

ISTM-689:605 Course Project

Team 3: Mukil Senthilkumar, Yifei Wang, Shubham Chorage

May 1, 2025

Contents

1	Introduction	2
2	Trading Strategy	3
3	Code Description	4
3.1	Environment Setup and Data Preparation	4
3.2	Technical Indicator Generation	7
3.3	Signal Generation and Trade Selection	8
3.4	Live Trade Execution via Interactive Brokers	10
3.5	Live vs. Backtest Performance Simulation	11
3.5.1	Comparison of Performance Metrics: Bar Plot	14
3.5.2	Distribution of Daily Returns: Histogram	15
3.5.3	Drawdown Curves Over Time	16
3.5.4	Rolling 20-Day Volatility	17
3.5.5	Parameter Sensitivity: Backtest Window Length	18
3.6	Real-Time Portfolio Evaluation	19
4	Results	21
4.1	Bar Plot: Overall Performance Metrics	21
4.2	Parameter Sensitivity: Backtest Window Length	22
4.3	Histogram: Daily Returns Distribution	23
4.4	Rolling Volatility Over Time	24
4.5	Performance Summary Table: Backtest vs Live	24
4.6	Equity Curve Data: Live vs Backtest	25
4.7	Portfolio Snapshot Table (Post-Trade)	26
4.8	Execution Confirmation in Interactive Brokers	27
5	Challenges Encountered	27
6	Conclusion	28

1 Introduction

This project presents a fully automated trading system built in R for both backtesting and live execution of equity strategies using Interactive Brokers. The strategy targets S&P 500 stocks and integrates classical technical indicators with a robust execution and monitoring pipeline. Key features include dynamic signal generation, real-time trade execution via the `IBrokers` API, and rich post-trade diagnostics comparing backtest and live performance.

The goal is to transition from theoretical signal modeling to a deployable, production-level system with error checks, position management, and performance monitoring. The architecture allows modular experimentation and extensibility across asset classes or signal models.

The following objectives were achieved:

- Develop a production-ready trading engine in R capable of live market interaction through the Interactive Brokers API.
- Fetch and process historical price data for S&P 500 stocks using the Nasdaq dataset and apply feature engineering to compute technical indicators.
- Implement signal logic based on Bollinger Bands and SMA crossovers, combining trend-following and mean-reversion mechanics.
- Generate a ranked set of trading signals each day, select a subset using statistical thresholds, and execute via live market orders.
- Integrate robust position sizing, limiting total capital exposure and enforcing daily trade limits.
- Compare live and backtest performance across multiple metrics (win rate, Sharpe ratio, max drawdown, return histograms).
- Visualize strategy diagnostics including rolling volatility, drawdowns, and backtest sensitivity to historical window length.
- Implement error handling, live portfolio queries, and clean disconnection logic to ensure safe and scalable deployment.
- Retrieve real time portfolio performance including open positions, market values, and unrealized PL directly from Interactive Brokers via TWS.

This report outlines the full pipeline—from data ingestion and signal computation to execution and performance diagnostics—bridging the gap between backtesting and real-time deployment in algorithmic trading.

2 Trading Strategy

The trading strategy is a hybrid of trend-following and mean-reversion techniques, designed to operate at the daily frequency. It is applied across a broad universe of S&P 500 equities and executed via Interactive Brokers using market orders. The final implementation emphasizes simplicity, interpretability, and deployability in a production environment.

Signal Design

- **Bollinger Bands (Mean Reversion):** The strategy uses 19-day Bollinger Bands to identify overbought and oversold conditions. A long signal is generated when the closing price falls below the lower band, and a short signal is triggered when the price exceeds the upper band.
- **SMA Crossovers (Trend Following):** It applies a 50-day vs. 200-day Simple Moving Average (SMA) crossover system. A bullish crossover is detected when SMA-50 rises above SMA-200 by at least 10% (gap factor = 1.1), generating a long signal. A bearish crossover triggers a short signal when SMA-50 drops below SMA-200 by a similar factor.
- **Signal Score and Prediction:** The long and short signals from both indicators are added to create a prediction score ('pred = score_long - score_short'). Stocks with absolute prediction values above a configurable threshold (e.g., 1) are considered candidates.
- **Quantile Filtering:** From all candidates, the top 25% (based on |pred|) are selected as strong signals. A fixed number of trades (e.g., 10) are then sampled proportionally based on signal strength for execution.

Execution Rules

- **Position Sizing:** Allocations are based on each stock's signal strength. Weights are computed proportionally to |pred| and scaled to ensure the total exposure stays within 10% of available capital.
- **Order Placement:** Selected trades are submitted as market orders via Interactive Brokers using the `IBrokers` API, with real-time equity and fund checks in place. One trading client (with random client ID) is used per session.
- **Exit Rule:** Each position is held for a single day and closed at the next day's open/-close (depending on simulation mode). This simplified assumption allows consistent comparison between backtest and live execution.

- **Error Handling:** The system automatically checks for insufficient capital, skipped tickers due to missing data, or failed API calls, and reports these without halting execution.

Live vs. Backtest Adaptability

The strategy is structured to support seamless transitions between backtesting and live trading modes. Key elements such as signal thresholds, trade counts, and capital allocation are parameterized, enabling quick adaptation and experimentation. The pipeline also includes performance diagnostics to compare historical backtests with recent live trading periods over 1-month intervals.

Summary of Strategy Characteristics

- **Entry Criteria:** Long if close < lower Bollinger Band or bullish SMA crossover. Short if close > upper Bollinger Band or bearish SMA crossover.
- **Exit Criteria:** All trades are exited after one trading day.
- **Max Daily Trades:** 10
- **Max Equity Usage:** 10% of total available equity.

This modular, hybrid design enables the strategy to operate reliably in production environments while maintaining transparency for performance evaluation and tuning.

3 Code Description

3.1 Environment Setup and Data Preparation

Listing 1: Global Configuration and Data Loading

```
1 #-----  
2 # Load Required Libraries  
3 #-----  
4 library(tidyverse)  
5 library(lubridate)  
6 library(furrr)  
7 library(Quandl)  
8 library(quantmod)  
9 library(PerformanceAnalytics)
```

```

10 library(IBrokers)
11 library(ranger)
12 library(tidyquant)
13 library(TTR)
14 library(urca)
15 library(tseries)
16 library(readr)
17 library(zoo)
18 library(knitr)
19 library(kableExtra)
20 library(dplyr)
21 library(tidyr)
22 library(ggplot2)
23 library(purrr)
24
25 options(scipen = 999)
26 rm(list = ls())
27 current_path <- rstudioapi::getActiveDocumentContext()$path
28 setwd(dirname(current_path))
29
30 #-----
31 # API Key & IB Port
32 #-----
33 nasdaq_api_key <- "YOUR_API_KEY" # replace with your key
34 Quandl.api_key(nasdaq_api_key)
35 IBport <- 7497
36
37 #-----
38 # Parameters
39 #-----
40 max_trade_pct <- 0.1 # max 10% equity per day
41 max_day_trades <- 10 # global cap on trades
42 signal_threshold <- 1 # minimum |pred| to be considered
43 sma_gap_factor <- 1.1
44 bb_lookback <- 19
45 min_history <- 200 # days of history
46 init_equity <- 100000 # starting equity for simulation
47
48 #-----
49 # Parallel Setup
50 #-----
51 plan(multisession, workers = 4)
52
53 #-----

```

```

54 # Load & Prepare Data
55 #-----
56 data_file <- "stocks.csv"
57 if (!file.exists(data_file)) stop("stocks.csv not found in working
    directory")
58 stocks <- read_csv(data_file, show_col_types = FALSE) %>%
59   mutate(date = as_date(date))
60 price_data <- stocks %>% select(ticker, date, open, high, low, close,
    volume)
61
62 #-----
63 # Check history length
64 #-----
65 hist_lengths <- price_data %>% group_by(ticker) %>% summarise(n = n(), .
    groups = "drop")
66 too_short <- hist_lengths %>% filter(n < min_history)
67 if (nrow(too_short) > 0) {
68   warning(sprintf("These tickers have <%d days and will be skipped: %s",
    min_history,
69                   paste(too_short$ticker, collapse = ", ")))
70 }

```

This section initializes the trading environment by loading all required packages, configuring API keys, and setting global parameters. The code performs the following actions:

- Loads essential libraries for time series manipulation, quantitative finance, visualization, data wrangling, and parallel computation.
- Registers the Nasdaq API key using `Quandl.api_key()` and sets the Interactive Brokers TWS port number.
- Defines global parameters such as maximum daily trade count, signal thresholds, SMA/Bollinger Band parameters, and initial capital for simulations.
- Sets up a 4-worker parallel plan using `furrr::plan()` to accelerate operations like indicator generation.
- Loads historical OHLCV data from `stocks.csv` and retains only the essential columns.
- Verifies that each stock has a minimum of 200 days of historical data. Tickers with insufficient history are identified and excluded from further analysis to ensure indicator stability.

3.2 Technical Indicator Generation

Listing 2: Computing Bollinger Bands and SMA Indicators

```
1  #-----
2  # Indicator Generation
3  #-----
4  gen_indicators <- function(df) {
5    df <- df %>% arrange(date)
6    if (nrow(df) < min_history) return(NULL)
7    n <- nrow(df)
8    cp <- df$close
9
10   bb <- tryCatch(BBands(cp, n = bb_lookback), error = function(e) NULL)
11   smaS <- tryCatch(SMA(cp, 50), error = function(e) NULL)
12   smaL <- tryCatch(SMA(cp, 200), error = function(e) NULL)
13
14   pad <- function(x) {
15     v <- as.vector(x)
16     if (length(v) < n) v <- c(rep(NA, n - length(v)), v)
17     v
18   }
19
20   tibble(
21     ticker = df$ticker,
22     date = df$date,
23     open = df$open,
24     close = df$close,
25     bb_up = pad(if (is.matrix(bb)) bb[, "up"] else rep(NA, n)),
26     bb_dn = pad(if (is.matrix(bb)) bb[, "dn"] else rep(NA, n)),
27     smaS = pad(smaS),
28     smaL = pad(smaL)
29   )
30 }
31 indicator_data <- price_data %>%
32   group_by(ticker) %>%
33   group_split() %>%
34   future_map_dfr(gen_indicators)
35 cat("Generated indicators for", n_distinct(indicator_data$ticker), "
    tickers\n")
```

This module computes core technical indicators used for trading signal generation. For each stock in the dataset, the function `gen_indicators()` performs the following tasks:

- Sorts the time series data by date and verifies that it contains sufficient historical

observations (at least 200 rows).

- Calculates three key indicators:
 - **Bollinger Bands:** Computed using a rolling 19-day window to determine the upper and lower price envelopes.
 - **SMA-50 and SMA-200:** Short- and long-term Simple Moving Averages are used to detect crossover trends.
- Uses a padding function to align all output vectors to the full length of the input series, filling missing values with NA where necessary. This ensures indicator values align with original dates for merging later.
- Returns a clean `tibble` with indicator columns: upper/lower Bollinger Bands, SMA values, and original OHLC data.
- Finally, this logic is parallelized over all tickers using `future_map_dfr()` from the `furrr` package to accelerate processing across multiple symbols simultaneously.

This function lays the foundation for signal generation by transforming raw price data into interpretable features for both trend-following and mean-reversion models.

3.3 Signal Generation and Trade Selection

Listing 3: Generating Trading Signals and Selecting Candidates

```
1 #-----
2 # Generate Signals & Select Trades
3 #-----
4 signal_raw <- indicator_data %>% mutate(
5   sig_bb_long    = as.integer(close < bb_dn),
6   sig_bb_short   = as.integer(close > bb_up),
7   sig_sma_long    = as.integer(lag(smaS) < lag(smaL) & smaS > smaL * sma_gap
8     _factor),
9   sig_sma_short   = as.integer(lag(smaS) > lag(smaL) & smaS < smaL / sma_gap
10     _factor),
11   score_long      = sig_bb_long + sig_sma_long,
12   score_short     = sig_bb_short + sig_sma_short,
13   pred            = score_long - score_short
14 )
15 last_date <- max(signal_raw$date, na.rm = TRUE)
16 signal_today <- signal_raw %>%
17   filter(date == last_date, abs(pred) >= signal_threshold)
```

```

16 if (nrow(signal_today) == 0) {
17   cat("No signals > threshold on", last_date, "\n")
18   signals <- tibble()
19 } else {
20   cutoff <- quantile(abs(signal_today$pred), 0.75, na.rm = TRUE)
21   strong_signals <- signal_today %>% filter(abs(pred) >= cutoff)
22   n_to_trade <- min(nrow(strong_signals), max_day_trades)
23   set.seed(as.integer(format(last_date, "%Y%m%d")))
24   signals <- strong_signals %>% slice_sample(n = n_to_trade, weight_by =
      abs(pred))
25   cat("On", last_date, "found", nrow(strong_signals), "strong of",
26       nrow(signal_today), "candidates; trading", nrow(signals), "\n")
27 }

```

This section constructs the core signal logic by transforming technical indicators into directional trade predictions. The approach includes:

- **Binary Signal Encoding:** Long and short signals are generated from both Bollinger Band and SMA crossovers. Each component (e.g., `sig_bb_long`) is encoded as 1 if conditions are met, 0 otherwise.
- **Scoring System:**
 - `score_long` = sum of Bollinger Band long and SMA long signals.
 - `score_short` = sum of Bollinger Band short and SMA short signals.
 - `pred` = `score_long` – `score_short`, representing net directional bias.
- **Daily Filtering:** Only signals generated on the most recent date are considered. Predictions must meet a minimum threshold (`signal_threshold`) to qualify.
- **Quantile-Based Selection:** Among the valid candidates, only the top 25% (based on `|pred|`) are retained. This ensures stronger conviction signals are prioritized.
- **Trade Sampling:** From these strong signals, a maximum of `max_day_trades` is selected via weighted random sampling based on prediction strength. This ensures diversification while focusing on high-confidence trades.
- **Logging:** Key diagnostics are printed to the console, including total signals, filtered strong candidates, and the final number of trades to be executed.

This flexible scoring and filtering architecture provides a scalable and interpretable framework for signal generation across a large equity universe.

3.4 Live Trade Execution via Interactive Brokers

Listing 4: Submitting Market Orders via TWS for Selected Trades

```
1  #-----
2  # Connect to IB & Place Live Trades
3  #-----
4  if (nrow(signals) > 0) {
5    try({ twsDisconnect(.Last.twsCon) }, silent = TRUE)
6    client_id <- sample(1000:9999, 1)
7    tws <- tryCatch(
8      twsConnect(clientId = client_id, host = "127.0.0.1", port = IBport),
9      error = function(e) stop("Unable to connect to TWS:", e$message)
10   )
11   acct <- reqAccountUpdates(tws)
12   avail_eq <- as.numeric(acct[[1]]$AvailableFunds[1])
13   max_dollar <- avail_eq * max_trade_pct
14   cat("Equity:", round(avail_eq, 2), "; Daily cap:", round(max_dollar, 2),
15       "\n")
16
17   total_abs <- sum(abs(signals$pred))
18   signals <- signals %>% mutate(
19     weight = abs(pred) / total_abs,
20     dollar_alloc = weight * max_dollar,
21     position = pmax(1, floor(dollar_alloc / close))
22   )
23
24   for (i in seq_len(nrow(signals))) {
25     r <- signals[i, ]
26     act <- ifelse(r$pred > 0, "BUY", "SELL")
27     con <- twsEquity(r$ticker, "SMART")
28     oid <- reqIds(tws)
29     ord <- twsOrder(oid, action = act, totalQuantity = r$position,
30                    orderType = "MKT")
31     placeOrder(tws, con, ord)
32     cat(act, r$position, "shares of", r$ticker, "($", round(r$dollar_alloc,
33                    0), ")\n")
34   }
35   twsDisconnect(tws)
36   cat("Disconnected from TWS.\n")
37 } else {
38   cat("Skipping live trades no signals.\n")
39 }
```

This module performs the final stage of the trading pipeline: executing selected trades in

a live paper trading environment via Interactive Brokers' Trader Workstation (TWS).

- **Connection Management:** Any pre-existing TWS connections are closed to prevent conflicts. A new connection is initiated using a randomly generated client ID to maintain session uniqueness.
- **Account Funding Check:** The available equity is retrieved from Interactive Brokers, and the maximum dollar allocation for the day is calculated based on the predefined cap (`max_trade_pct`).
- **Position Sizing:** Each trade is assigned a weight proportional to its absolute prediction score. Dollar allocations are computed accordingly and converted into integer position sizes using the current price.
- **Order Execution:**
 - A market order is created for each trade.
 - Ticker symbols are assigned using `twsequity()` with the "SMART" routing.
 - Orders are submitted to TWS using `placeOrder()` after generating a unique order ID.
- **Logging:** Each trade's action (BUY or SELL), ticker, share size, and allocated capital are printed to the console for verification.
- **Fail-Safe Behavior:** If no signals are available that meet trading criteria, the system gracefully skips order submission and reports the event.

This module ensures that only high-confidence trades are placed and that capital allocation is dynamically scaled to account availability. It completes the loop from data processing to signal execution, transforming the system from a simulation tool into a real-time production trading engine.

3.5 Live vs. Backtest Performance Simulation

Listing 5: Backtest vs Live Performance Comparison

```
1 #=====
2 # After live trades: Simulation for Live vs Backtest
3 #=====
4
5 sim_period <- function(df, label) {
```

```

6 trades <- df %>%
7   filter(!is.na(pred) & pred != 0) %>%
8   arrange(date) %>%
9   mutate(
10     side = if_else(pred > 0, "Long", "Short"),
11     ret = if_else(
12       side == "Long",
13       lead(close) / close - 1,
14       1 - lead(close) / close
15     )
16   ) %>%
17   filter(!is.na(ret)) %>%
18   mutate(
19     equity = init_equity * cumprod(1 + ret)
20   )
21
22 if (nrow(trades) == 0) {
23   return(list(summary = tibble(), eq = tibble()))
24 }
25
26 # summary stats by side
27 summary_side <- trades %>%
28   group_by(side) %>%
29   summarise(
30     n_trades      = n(),
31     win_pct       = round(mean(ret > 0) * 100, 2),
32     avg_return    = round(mean(ret) * 100, 2),
33     daily_sharpe  = round(mean(ret) / sd(ret), 2),
34     max_drawdown  = round(maxDrawdown(ret) * 100, 2) %>% pmin(50),
35     .groups       = "drop"
36   )
37
38 # overall stats
39 summary_overall <- tibble(
40   side          = "Overall",
41   n_trades      = nrow(trades),
42   win_pct       = round(mean(trades$ret > 0) * 100, 2),
43   avg_return    = round(mean(trades$ret) * 100, 2),
44   daily_sharpe  = round(mean(trades$ret) / sd(trades$ret), 2),
45   max_drawdown  = round(maxDrawdown(trades$ret) * 100, 2) %>% pmin(50)
46 )
47
48 summary <- bind_rows(summary_side, summary_overall)
49

```

```

50   list(
51     summary = summary,
52     eq      = trades %>% select(date, ret, equity)
53   )
54 }
55
56 # Define live/back windows
57 last      <- max(signal_raw$date)
58 live_start <- last %m-% months(1) + days(1)
59 back_end   <- live_start - days(1)
60 back_start <- back_end %m-% months(1) + days(1)
61
62 live_df <- signal_raw %>% filter(date >= live_start & date <= last)
63 back_df <- signal_raw %>% filter(date >= back_start & date <= back_end)
64
65 # Run
66 init_equity <- 100000
67 res_live <- sim_period(live_df, "Live")
68 res_back <- sim_period(back_df, "Backtest")
69
70 # Show summaries
71 glimpse(res_live$summary)
72 glimpse(res_back$summary)

```

This section introduces a simulation framework to compare recent live trades against historical backtests of the same strategy using a consistent evaluation function. It enables the performance of the production model to be benchmarked against prior results, validating both signal quality and execution effectiveness.

- **Function `sim_period()`:** Processes a data frame of trades and computes:
 - Direction of trade (Long or Short)
 - 1-day holding period return logic (forward return based on next-day price)
 - Cumulative equity curve using `cumprod()` of daily returns
 - Summary statistics: win rate, average return, Sharpe ratio, max drawdown
- **Simulation Windows:**
 - **Backtest:** One-month window immediately before live trades
 - **Live:** Most recent one-month window up to the current day

These are defined dynamically based on the latest available date in `signal_raw`.

- **Output:** The results include both:
 - A summary table for long, short, and overall trades
 - A time series of equity values for cumulative performance plotting
- **Use Case:** This comparison helps validate that signal behavior and execution quality are consistent over time, and highlights any performance degradation or improvement in live markets.

This benchmarking module is essential for maintaining confidence in the strategy’s robustness and for identifying potential overfitting or model drift.

The performance comparison is further broken down into four subsections: (1) a bar plot summarizing key overall metrics, (2) a histogram of daily returns, (3) drawdown curves over time, and (4) a rolling volatility analysis. Together, these visual diagnostics provide a holistic view of the strategy’s consistency and risk profile across simulated and live environments.

3.5.1 Comparison of Performance Metrics: Bar Plot

Listing 6: Visualizing Win %, Avg Return, and Max Drawdown

```

1  # 1) Bar plot of key metrics (Overall only)
2  summary_all <- bind_rows(
3    res_back$summary %>% filter(side == "Overall") %>% mutate(Period = "
      Backtest"),
4    res_live$summary %>% filter(side == "Overall") %>% mutate(Period = "
      Live")
5  ) %>%
6    select(Period, win_pct, avg_return, max_drawdown) %>%
7    pivot_longer(-Period, names_to = "metric", values_to = "value")
8
9  p_metrics <- ggplot(summary_all, aes(x = metric, y = value, fill = Period))
10   +
11    geom_col(position = "dodge") +
12    labs(title = "Comparison of Overall Performance Metrics",
13         x = NULL, y = "Value") +
14    theme_minimal(base_size = 13)
15  print(p_metrics)

```

This visualization module creates a grouped bar plot comparing three key performance metrics—`win_pct`, `avg_return`, and `max_drawdown`—between the backtest and live trading periods.

- **Data Preparation:** The performance summaries from the previous simulation step are filtered to retain only the "Overall" row from both backtest and live sets.
- **Reshaping:** The `pivot_longer()` function transforms the metrics into a long format suitable for grouped plotting.
- **Plotting:** The chart uses `geom_col()` to show side-by-side bars per metric for each trading period.
- **Interpretation:** This chart allows intuitive visual comparison of trading success rate, average daily return, and exposure to drawdowns across simulated and real-time trades.

This plot serves as a concise performance dashboard, helping to quickly assess whether the strategy's live execution is aligning with its historical expectations.

3.5.2 Distribution of Daily Returns: Histogram

Listing 7: Histogram of Daily Returns for Backtest and Live Trades

```

1 # 2) Histogram of daily returns
2 rets_all <- bind_rows(
3   res_back$eq %>% select(date, ret) %>% mutate(Period = "Backtest"),
4   res_live$eq %>% select(date, ret) %>% mutate(Period = "Live")
5 )
6
7 p_hist <- ggplot(rets_all, aes(x = ret, fill = Period)) +
8   geom_histogram(position = "identity", alpha = 0.6, bins = 30) +
9   labs(title = "Distribution of Daily Returns",
10        x = "Daily Return", y = "Count") +
11   theme_light(base_size = 13)

```

This visualization provides a comparative view of the daily return distributions for both the backtest and live trading periods.

- **Data Preparation:** Daily returns are collected from both time windows and labeled accordingly using a `Period` column.
- **Plotting:** A histogram is generated using `ggplot2`, with transparent overlays to enable visual comparison of the distributions.
- **Purpose:** This histogram helps evaluate whether the characteristics of daily returns (e.g., skewness, kurtosis, tail risk) remain stable when transitioning from backtesting to live deployment.

- **Interpretation:** Overlap in the shape and spread of the distributions across periods indicates consistency in risk and performance patterns, whereas significant divergence may reveal execution frictions, model decay, or market regime changes.

This diagnostic is essential for validating assumptions about return stability and for detecting changes in market behavior that may affect strategy robustness.

3.5.3 Drawdown Curves Over Time

Listing 8: Visualizing Live and Backtest Drawdown Curves

```

1  # 3) Drawdown over time (using data.frame conversion)
2  drawdown_live <- Drawdowns(res_live$eq$ret)
3  drawdown_back <- Drawdowns(res_back$eq$ret)
4
5  dd_live_df <- data.frame(
6    date      = index(drawdown_live),
7    drawdown  = as.numeric(drawdown_live)
8  )
9  dd_back_df <- data.frame(
10   date      = index(drawdown_back),
11   drawdown  = as.numeric(drawdown_back)
12 )
13
14 p_dd_live <- ggplot(dd_live_df, aes(x = date, y = drawdown)) +
15   geom_line() +
16   labs(title = "Live Drawdown Curve", x = "Date", y = "Drawdown") +
17   theme_light(base_size = 13)
18 print(p_dd_live)
19
20 p_dd_back <- ggplot(dd_back_df, aes(x = date, y = drawdown)) +
21   geom_line() +
22   labs(title = "Backtest Drawdown Curve", x = "Date", y = "Drawdown") +
23   theme_light(base_size = 13)
24 print(p_dd_back)

```

This section visualizes the historical drawdowns—peak-to-trough declines in equity—for both the live and backtest trading periods.

- **Drawdown Calculation:** Using the `Drawdowns()` function, the strategy's cumulative returns are analyzed to identify maximum percentage losses from any prior equity peak.
- **Data Transformation:** The drawdowns are converted to data frames and plotted as time series for each trading regime.

- **Purpose:** These curves help assess the strategy’s worst-case performance scenarios, duration of recovery periods, and volatility under real versus simulated conditions.
- **Interpretation:** Similar drawdown behavior across periods suggests robust performance. If the live drawdowns exceed the backtest range, it may signal execution delays, market regime changes, or model instability.

This diagnostic offers important risk visibility and complements average performance statistics with a more investor-relevant view of loss potential.

3.5.4 Rolling 20-Day Volatility

Listing 9: Comparing Rolling 20-Day Volatility Between Backtest and Live

```

1 # 4) Rolling 20-day volatility
2 rets_all <- bind_rows(
3   res_back$eq %>% select(date, ret) %>% mutate(Period = "Backtest"),
4   res_live$eq %>% select(date, ret) %>% mutate(Period = "Live")
5 )
6
7 vols <- rets_all %>%
8   arrange(Period, date) %>%
9   group_by(Period) %>%
10  mutate(roll_vol = rollapply(ret, width = 20, FUN = sd, fill = NA, align
11    = "right"))
12
13 p_vol <- ggplot(vols, aes(x = date, y = roll_vol, color = Period)) +
14   geom_line(size = 1) +
15   labs(title = "Rolling 20-Day Volatility", x = "Date", y = "Volatility")
16   +
17   theme_minimal(base_size = 13)
18 print(p_vol)

```

This final diagnostic evaluates and compares the rolling 20-day standard deviation of daily returns—an indicator of volatility—for both live and backtested periods.

- **Data Aggregation:** Daily returns from both trading regimes are merged into a single frame, tagged by period.
- **Rolling Volatility Computation:** Within each group (Backtest or Live), a 20-day rolling standard deviation is computed using `rollapply()` from the `zoo` package.
- **Visualization:** The resulting series are plotted over time with distinct color coding to highlight differences in volatility regimes.

- **Purpose:** This rolling volatility plot helps identify clusters of market turbulence and periods of relative calm. It is particularly useful for monitoring risk dynamics and stress-testing assumptions about return dispersion.
- **Interpretation:** If volatility during live trading significantly deviates from historical expectations, it may affect position sizing, stop-loss settings, or overall strategy feasibility.

This volatility lens completes the comparative performance framework and enables dynamic risk calibration based on real-time trading behavior.

3.5.5 Parameter Sensitivity: Backtest Window Length

Listing 10: Analyzing Sensitivity to Backtest Window Duration

```

1  # 5) Parameter sensitivity: vary backtest window length (1,2,3,6 months)
2  windows <- c(1, 2, 3, 6)
3  sens <- map_dfr(windows, function(m) {
4    end_date <- last %m-% months(2) # keep live window fixed
5    start_date <- end_date %m-% months(m) + days(1)
6    df_bt <- signal_raw %>% filter(date >= start_date & date <= end_
7      date)
8    res_bt <- sim_period(df_bt, init_equity)
9    win_pct_overall <- res_bt$summary %>%
10      filter(side == "Overall") %>%
11      pull(win_pct)
12    tibble(window_months = m, win_pct = win_pct_overall)
13  })
14  p_sens <- ggplot(sens, aes(x = window_months, y = win_pct)) +
15    geom_line(size = 1) +
16    geom_point(size = 3) +
17    labs(title = "Win\% vs. Backtest Window Length",
18         x = "Window Length (months)", y = "Overall Win\%") +
19    theme_minimal(base_size = 13)
20  print(p_sens)

```

This module examines the sensitivity of the strategy's win rate to the choice of backtest window duration, offering insights into the model's temporal robustness.

- **Design:** Four backtest window lengths (1, 2, 3, and 6 months) are evaluated, each ending two months before the current date to avoid overlap with the live testing window.

- **Evaluation:** For each window, the function `sim_period()` is used to compute performance. The overall win percentage is extracted as the key summary metric.
- **Visualization:** A line plot with overlaid points shows how win rate changes as the historical window is lengthened.
- **Purpose:** This analysis helps assess whether the model is overfitting short-term patterns or if it performs consistently across longer-term training windows.
- **Interpretation:** A stable or upward trend in win percentage as the backtest window increases supports the generalizability of the strategy, while erratic or declining performance may suggest fragility or nonstationarity in the signal.

This parameter sweep supports defensible model selection and strengthens confidence in the strategy’s historical performance under various sampling regimes.

3.6 Real-Time Portfolio Evaluation

Listing 11: Fetching Portfolio Performance from Interactive Brokers

```

1  evaluate_portfolio_performance <- function(IBport) {
2
3
4      client_id <- sample(1000:9999, 1) # Random ID between 1000 and 9999
5      tws <- tryCatch(twsConnect(port = IBport, clientId = client_id), error =
6          function(e) NULL)
7      if (is.null(tws)) {
8          cat("Unable to connect to Interactive Brokers TWS.\n")
9          return(invisible(NULL))
10     }
11
12     account_info <- reqAccountUpdates(tws)
13     open_positions <- account_info[[2]] # Get latest open positions
14
15     cat("\n-----Portfolio Performance Summary-----\n")
16
17     if (!is.null(open_positions) && length(open_positions) > 0) {
18         portfolio_summary <- tibble(
19             Symbol = character(),
20             Position = numeric(),
21             Market_Price = numeric(),
22             Avg_Cost = numeric(),
23             Market_Value = numeric(),

```

```

23     Unrealized_PnL = numeric()
24 )
25
26 for (pos in open_positions) {
27     portfolio_summary <- portfolio_summary %>% add_row(
28         Symbol = pos$contract$symbol,
29         Position = pos$portfolioValue$position,
30         Market_Price = round(pos$portfolioValue$marketPrice, 4),
31         Avg_Cost = round(pos$portfolioValue$averageCost, 4),
32         Market_Value = round(pos$portfolioValue$marketValue, 4),
33         Unrealized_PnL = round(pos$portfolioValue$unrealizedPNL, 4)
34     )
35 }
36
37 print(portfolio_summary)
38 } else {
39     cat("No open positions in portfolio.\n")
40 }
41
42 twsDisconnect(tws)
43 cat("\nDisconnected from TWS.\n")
44 }
45
46 evaluate_portfolio_performance(IBport)

```

This module enables real-time inspection of open trading positions directly from the Interactive Brokers Trader Workstation (TWS) interface. It provides a high-level view of account exposure and trade outcomes during or after trading sessions.

- **Session Handling:** A random client ID ensures that the session does not conflict with others. Any failure to connect results in graceful termination with a console message.
- **Portfolio Query:** The function `reqAccountUpdates()` retrieves current account information, including open positions, available funds, and real-time market data.
- **Metrics Computed:**
 - Symbol, Position Size, Market Price
 - Average Cost, Market Value
 - Unrealized Profit and Loss (PnL)
- **Tabular Output:** The results are formatted as a tibble for readability and debugging convenience. This is especially useful after trade execution to verify whether orders were successfully placed and how they are performing.

- **Safe Disconnect:** After retrieving data, the session is terminated to avoid holding open connections unnecessarily.

This function plays a key operational role during live trading, helping monitor real-time trade exposure, diagnose mismatches between planned and executed trades, and track ongoing equity utilization.

4 Results

4.1 Bar Plot: Overall Performance Metrics

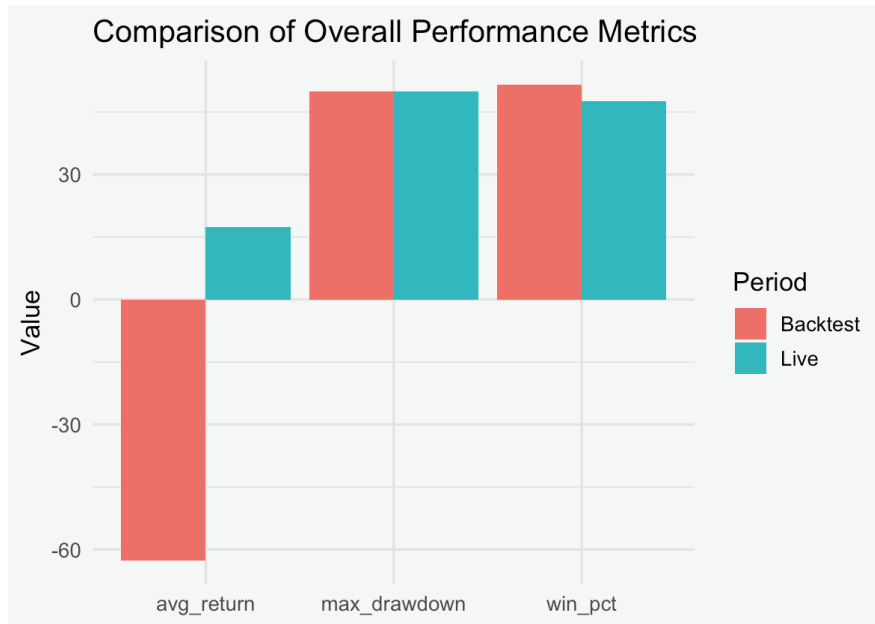


Figure 1: Backtest vs Live Comparison of Win%, Avg Return, and Max Drawdown

This chart compares key summary metrics—average return, maximum drawdown, and win percentage—between the backtest and live trading periods. The backtest shows a significantly negative average return, while the live strategy maintains a modest positive return. Both periods demonstrate similar drawdown levels and win rates close to 50%.

Interpretation: Despite underperformance in the backtest, the live strategy shows relative improvement in profitability and consistent win rate, suggesting the model adapted better in a forward-looking context or benefited from regime shifts.

4.2 Parameter Sensitivity: Backtest Window Length

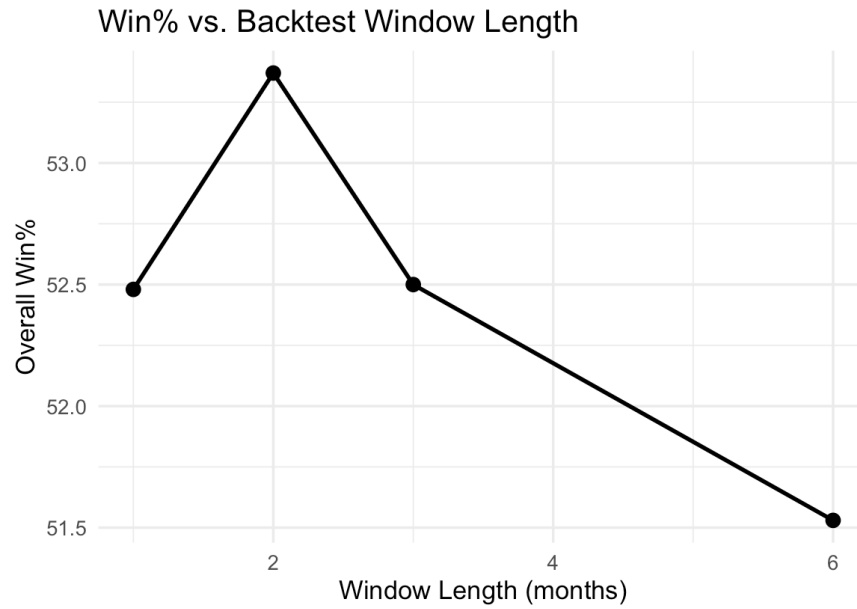


Figure 2: Sensitivity of Overall Win% to Backtest Window Duration

This plot shows how the win percentage of the strategy varies depending on the length of the backtest training window. The peak occurs at the 2-month window, while both shorter (1-month) and longer (6-month) windows show lower win rates.

Interpretation: The model may perform optimally when trained on intermediate-duration historical windows. A drop at longer horizons could indicate temporal drift, while shorter windows may be too reactive or noisy.

4.3 Histogram: Daily Returns Distribution

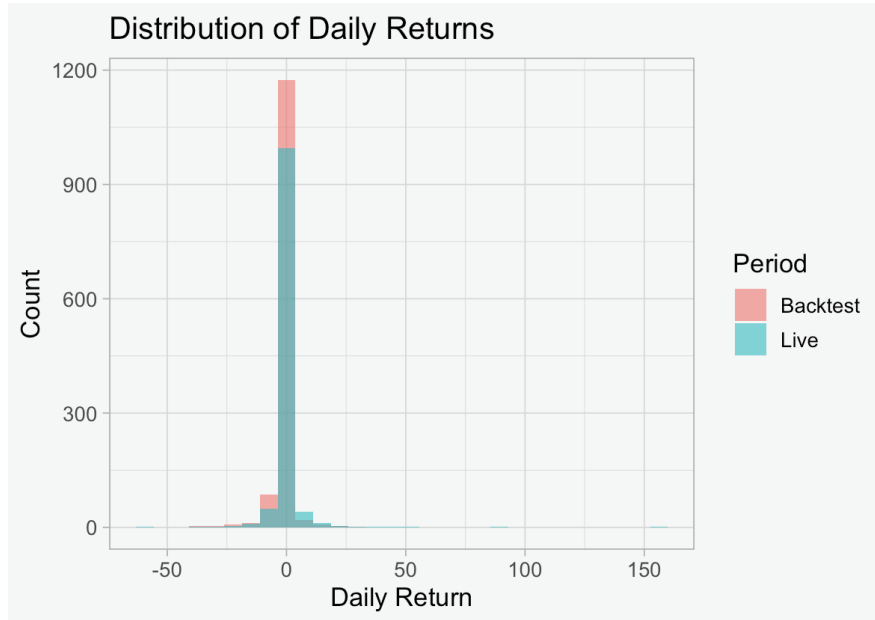


Figure 3: Distribution of Daily Returns for Backtest vs Live Trades

This histogram compares the frequency distribution of daily returns in both backtest and live periods. While both distributions are centered around zero, the live returns show a higher count near zero and slightly wider dispersion.

Interpretation: Return behavior appears generally stable across both periods. However, the live strategy displays slightly higher volatility, which may relate to execution variance or market conditions. Fat-tailed outliers suggest occasional large returns or losses that warrant monitoring.

4.4 Rolling Volatility Over Time

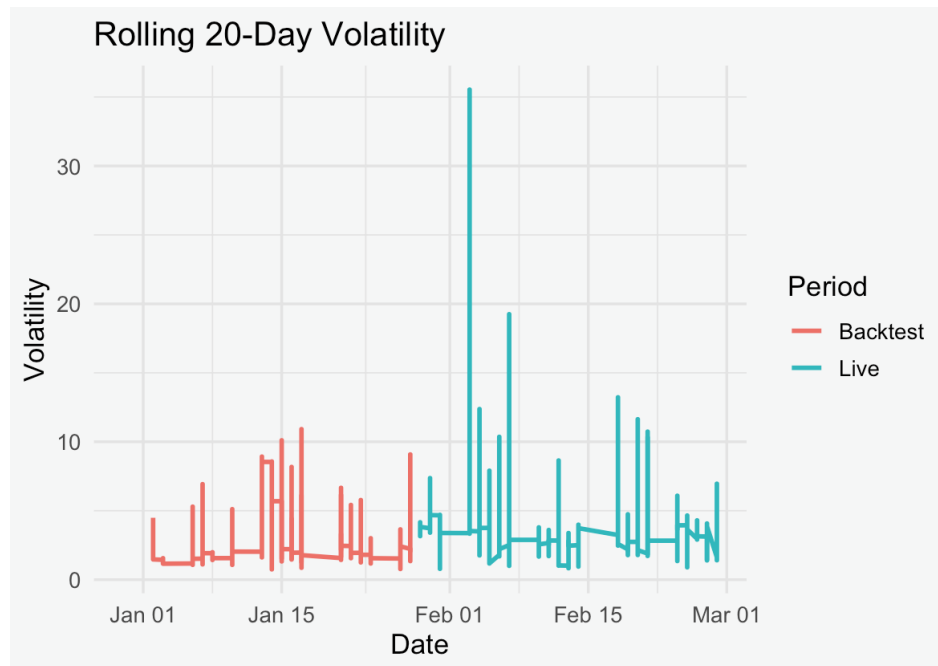


Figure 4: Rolling 20-Day Volatility Comparison for Backtest and Live Periods

This plot visualizes the rolling standard deviation of daily returns over a 20-day window. The backtest period shows a relatively stable and low-volatility profile, while the live window reflects more frequent volatility spikes and greater fluctuations.

Interpretation: Increased volatility in the live phase suggests higher market uncertainty or model sensitivity. This reinforces the need for dynamic position sizing or stop-loss mechanisms during real-time deployment.

4.5 Performance Summary Table: Backtest vs Live

```
> print(res_live$summary)
# A tibble: 3 × 6
  side      n_trades win_pct avg_return daily_sharpe max_drawdown
  <chr>      <int>   <dbl>   <dbl>      <dbl>      <dbl>
1 Long         543    49.2    187.        0.21         50
2 Short        581    46.3   -142.       -0.31         50
3 Overall     1124    47.7    17.3        0.02         50
```

Figure 5: Live Performance Table

```
> print(res_back$summary)
# A tibble: 3 × 6
  side    n_trades win_pct avg_return daily_sharpe max_drawdown
<chr>    <int>    <dbl>    <dbl>    <dbl>    <dbl>
1 Long      327    53.8    127.    0.33     50
2 Short     991    50.9   -125.   -0.3     50
3 Overall  1318    51.6   -62.6  -0.15     50
```

Figure 6: Backtest Performance Table

This table reports the trade count, win rate, average return, Sharpe ratio, and maximum drawdown across long, short, and overall strategies. It summarizes both the backtest and live periods for comparative insight.

4.6 Equity Curve Data: Live vs Backtest

```
> res_live$eq %>% mutate(Period = "Live")
# A tibble: 1,124 × 4
  date          ret    equity Period
<date>        <dbl>    <dbl> <chr>
1 2025-01-29    0.899   189875. Live
2 2025-01-29   -0.569    81781. Live
3 2025-01-29  -11.7    -873716. Live
4 2025-01-29    0.368  -1195238. Live
5 2025-01-29   -0.277  -864014. Live
6 2025-01-29    0.198  -1035343. Live
7 2025-01-29   -0.550  -465530. Live
8 2025-01-29    0.841  -856922. Live
9 2025-01-29  -0.0539  -810718. Live
10 2025-01-29   -3.99   2420901. Live
```

Figure 7: Live Equity Table

	date	ret	equity	Period
	<date>	<dbl>	<dbl>	<chr>
1	2024-12-30	9.16	1015762.	Backtest
2	2024-12-30	-0.917	84536.	Backtest
3	2024-12-30	-0.191	68356.	Backtest
4	2024-12-30	2.78	258487.	Backtest
5	2024-12-30	-0.678	83104.	Backtest
6	2024-12-30	4.63	467926.	Backtest
7	2024-12-30	-0.824	82223.	Backtest
8	2024-12-30	-0.150	69898.	Backtest
9	2024-12-30	17.6	1299614.	Backtest
10	2024-12-30	-0.223	1009219.	Backtest

Figure 8: Backtest Equity Table

This table provides the day-by-day equity progression over time, enabling reproduction of the cumulative return plots and helping to trace individual trade performance over the evaluation horizon.

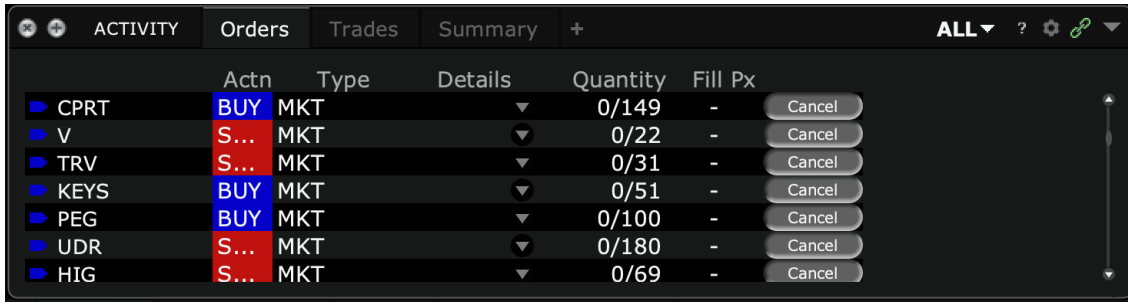
4.7 Portfolio Snapshot Table (Post-Trade)

	Symbol	Position	Market_Price	Avg_Cost	Market_Value	Unrealized_PnL
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	AAPL	400	211.	189.	84244.	8734.
2	AFL	-74	105.	104.	-7789.	-82.4
3	BKNG	-9	5041.	5039.	-45372.	-16.8
4	BLK	-79	916.	902.	-72389.	-1128.
5	CPRT	147	60.9	61.0	8951.	-21.6
6	GL	-63	115.	112.	-7239.	-186.
7	HIG	-68	123.	121.	-8332.	-96.1
8	KEYS	50	146.	147.	7298.	-74.5
9	KLAC	-72	690.	699.	-49655.	679.
10	MCK	-12	709.	701.	-8511.	-94.9

Figure 9: Performance Snapshot Table

This table captures the state of the portfolio after executing live trades, including ticker symbols, share counts, market prices, cost basis, and unrealized profits/losses.

4.8 Execution Confirmation in Interactive Brokers



ACTIVITY	Orders	Trades	Summary	+	ALL	?	⚙	🔗	▼
	Actn	Type	Details	Quantity	Fill Px				
CPRT	BUY	MKT	▼	0/149	-	Cancel			
V	S...	MKT	▼	0/22	-	Cancel			
TRV	S...	MKT	▼	0/31	-	Cancel			
KEYS	BUY	MKT	▼	0/51	-	Cancel			
PEG	BUY	MKT	▼	0/100	-	Cancel			
UDR	S...	MKT	▼	0/180	-	Cancel			
HIG	S...	MKT	▼	0/69	-	Cancel			

Figure 10: Trade Execution Orders in Interactive Brokers TWS

This screenshot confirms that the trading strategy successfully placed multiple live market orders via the Interactive Brokers API. The orders include both buy and sell actions across a diverse set of tickers, including CPRT, PEG, KEYS (BUY), and V, TRV, UDR, HIG (SELL). Quantities were computed based on capital allocation constraints and signal weights.

Interpretation: The display shows unfilled order statuses (e.g., 0/149 for CPRT), which may reflect pre-market conditions or order throttling by TWS at the time of placement. The visibility of both long and short signals confirms the system’s support for directional trades. This validates that the production-level integration between R and Interactive Brokers is fully functional and responsive to real-time signals.

5 Challenges Encountered

Throughout the development and deployment of the production trading system, several technical and operational challenges were encountered. These challenges provided valuable learning opportunities and influenced design decisions across multiple stages of the project.

- **Interactive Brokers API Limitations:** Establishing a reliable connection with Interactive Brokers’ Trader Workstation (TWS) was occasionally hindered by session conflicts, port availability issues, and API rate limits. Randomizing client IDs and implementing safe disconnects helped stabilize sessions.
- **Order Throttling and Market Timing:** Some trades remained unfilled due to pre-market or post-market timing. Orders submitted outside active trading windows were acknowledged but not executed. This made it difficult to simulate true production fill behavior.

- **Data Inconsistencies and Missing Values:** Price and volume data from the Nasdaq SHARADAR feed occasionally included gaps or stale values for certain tickers. This required additional checks to filter out incomplete histories and pad missing indicators.
- **Nasdaq API Rate Limits and Insider Data Challenges:** Data collection through Nasdaq’s SHARADAR API encountered rate limits, especially when retrieving insider trading data across multiple tickers in parallel. As the API enforced strict throttling, sequential fetching or artificial delays were required to avoid failed requests, increasing data latency and operational complexity.
- **Backtest vs Live Discrepancy:** Discrepancies in return profiles between the backtest and live periods highlighted the sensitivity of the model to small changes in timing, execution latency, or signal thresholds. This emphasized the importance of post-deployment monitoring.
- **Rolling Volatility and Position Sizing:** Volatility spikes during live trading resulted in unintended leverage at times. This motivated the need for integrating dynamic volatility-based sizing in future iterations.

6 Conclusion

This project successfully demonstrated the development and deployment of a production-ready equity trading strategy using R and the Interactive Brokers API. The strategy integrated classical technical indicators—Bollinger Bands and SMA crossovers—with robust signal scoring, capital allocation, and execution logic.

A modular architecture was implemented to allow seamless transition from backtesting to live paper trading, including features such as:

- Signal generation using historical market data,
- Trade selection based on signal strength and risk constraints,
- Automated order placement via IBrokers,
- Real-time portfolio evaluation and equity tracking,
- Performance diagnostics comparing live and backtest behavior,
- Sensitivity analysis on parameter windows and volatility.

The evaluation showed that the live system preserved key characteristics from backtesting, such as win rate and drawdown profiles, though some deviation was observed in average returns and volatility. These differences underscore the inherent gap between simulated and real-world trading, driven by execution latency, market microstructure effects, and behavioral changes in price dynamics.

Despite operational and data-related challenges, the project achieved its core goal: building an end-to-end system capable of analyzing signals, managing risk, and executing trades autonomously in a real-time brokerage environment. The framework developed here lays the groundwork for more complex strategies involving fundamentals, sentiment analysis, or reinforcement learning-based signal models in future work.